1. Build a Portfolio Page using HTML & CSS

SOURCE CODE:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.2/css/all.min.css">
  <link rel="stylesheet" href="index.css">
  <title>Portfolio Website</title>
</head>
<body>
  <header>
    <a href="#" class="logo">Glancy</a>
    <nav>
      <a href="#" class="active"> Home</a>
      <a href="#">Services</a>
      <a href="#">Skills</a>
      <a href="#">Education</a>
      <a href="#">Experience</a>
      <a href="#">Contact</a>
    </nav>
  </header>
  <section class="home">
    <div class="home-img">
      <img
src="https://t4.ftcdn.net/jpg/06/43/68/65/360_F_643686558_Efl6HB1ITw98bx1PdAd1wy56QpUTMh47
.jpg" alt="">
    </div>
    <div class="home-content">
      <h1>Hi, It's <span>Glancy</span></h1>
      <h3 class="typing-text">I'm a <span></span></h3>
      <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit. Minus labore dolores esse. Odit
similique doloribus tenetur doloremque, sunt commodi in ipsa repudiandae debitis deleniti blanditiis
quibusdam quaerat neque asperiores ea.</p>
      <div class="social-icons">
        <a href="#"><i class="fa-brands fa-linkedin"></i></a>
        <a href="#"><i class="fa-brands fa-github"></i></a>
        <a href="#"><i class="fa-brands fa-x-twitter"></i></a>
        <a href="#"><i class="fa-brands fa-instagram"></i></a>
      </div>
      <a href="#" class="btn">Hire me</a>
    </div>
  </section>
</body>
</html>
```

```
@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@100;200;300;400;500;6
00&display=swap');

* {

  margin: 0;

  padding: 0;

  box-sizing: border-box;

  text-decoration: none;

  border: none;

  outline: none;

  font-family: 'Poppins', sans-serif;

}

html {

  font-size: 62.5%;

}

body {

  width: 100%;

  height: 100vh;

  overflow-x: hidden;

  background-color: black;

  color: white;

}

header {

  margin-top: 20px;

  position: fixed;

  top: 0;
```

```
  left: 0;
  width: 100%;
  padding: 1rem 9%;
  background-color: transparent;
  filter: drop-shadow(10px);
  display: flex;
  justify-content: space-between;
  align-items: center;
  z-index: 100;
}
.logo {
  font-size: 3rem;
  color: #5377eb;
  font-weight: 800;
  cursor: pointer;
  transition: 0.5s ease;
}
.logo:hover {
  transform: scale(1.1);
}
nav a {
  font-size: 1.8rem;
  color: white;
  margin-left: 4rem;
  font-weight: 500;
```

```
  transition: 0.3s ease;
  border-bottom: 3px solid transparent;
}
nav a:hover,
nav a.active {
  color: #5377eb;
  border-bottom: 3px solid #5377eb;
}
@media(max-width:995px) {
  nav {
    position: absolute;
    display: none;
    top: 0;
    right: 0;
    width: 40%;
    border-left: 3px solid #5377eb;
    border-bottom: 3px solid #5377eb;
    border-bottom-left-radius: 2rem;
    padding: 1rem solid;
    background-color: #161616;
    border-top: 0.1rem solid rgba(0, 0, 0, 0.1);
  }

  nav.active {
    display: block;
```

```
  }
  nav a {
    display: block;
    font-size: 2rem;
    margin: 3rem 0;
  }
  nav a:hover,
  nav a.active {
  padding: 1rem;
    border-radius: 0.5rem;
    border-bottom: 0.5rem solid#5377eb;:
  }
}
section {
  min-height: 100vh;
  padding: 5rem 9% 5rem;
}
.home {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 8rem;
  background-color: black;
}
.home .home-content h1 {
```

```
  font-size: 6rem;
  font-weight: 700;
  line-height: 1.3;
}
span {
  color: #5377eb; ;
}
.home-content h3 {
  font-size: 4rem;
  margin-bottom: 1rem;
  font-weight: 700;
}
.home-content p {
  font-size: 1.6rem;
}
.home-img {
  border-radius: 50%;
}
.home-img img {
  position: relative;
  width: 32vw;
  border-radius: 50%;
  box-shadow: 0 0 25px solid#5377eb; ;
  cursor: pointer;
  transition: 0.2s linear;
```

```
}
.home-img img:hover {
  font-size: 1.8rem;
  font-weight: 500;
}
.social-icons a {
  display: inline-flex;
  justify-content: center;
  align-items: center;
  width: 4rem;
  height: 4rem;
  background-color:  transparent;
  border: 0.2rem solid#5377eb; ;
  font-size: 2rem;
  border-radius: 50%;
  margin: 3rem 1.5rem 3rem 0;
  transition: 0.3s ease;
  color: #5377eb;;
}
.social-icons a:hover {
  color: black;
  transform: scale(1.3) translateY(-5px);
  background-color: #5377eb;;
  box-shadow: 0 0 25px #5377eb;;
}
```

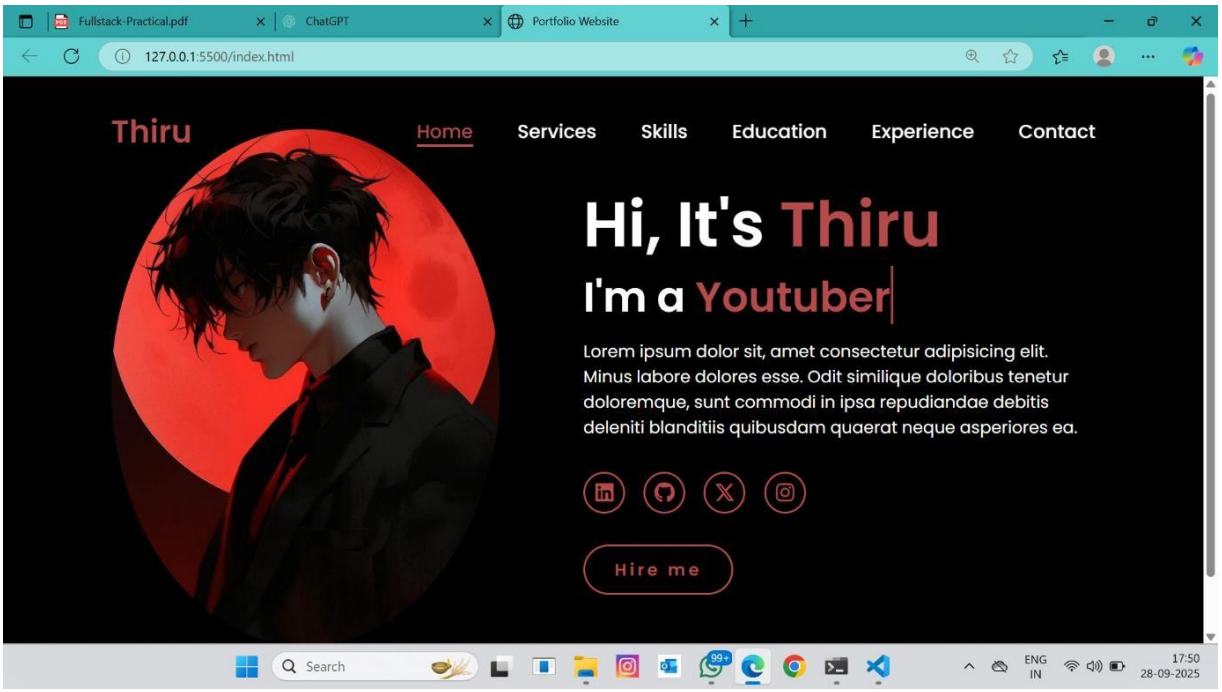
```
.btn {
  display: inline-block;
  padding: 1rem 2.8rem;
  background-color: black;
  border-radius: 4rem;
  font-size: 1.6rem;
  color: #5377eb; ;
  letter-spacing: 0.3rem;
  font-weight: 600;
  border: 2px solid#5377eb; ;
  transition: 0.3s ease;
  cursor: pointer;
}
.btn:hover {
  transform: scale3d(1.03);
  background-color: #5377eb; ;
  color: black;
  box-shadow: 0 0 25px#5377eb;;
}
.typing-text {
  font-size: 34px;
  font-weight: 600;
  min-width: 280px;
}
.typing-text span {
```

```
  position: relative;
}
.typing-text span::before {
  content: "software Developer";
  color: #5377eb;;
  animation: words 20s infinite;
}
.typing-text span::after {
  content: "";
  background-color: black;
  position: absolute;
  width: calc(100% + 8px);
  height: 100%;
  border-left: 3px solid black;
  right: -8;
  animation: cursor 0.6s infinite;
}
@keyframes cursor {
  to {
    border-left: 3px solid #5377eb;
  }
}
@keyframes words {
  0%,
  20% {
```

```
    content: "Web Developer";
  }
  21%,
  40% {
    content: "Developer";
  }
  41%,
  60% {
    content: "Web Designer";
  }
  61%,
  80% {
    content: "Youtuber";
  }
  81%,
  100% {
    content: "Script Writer";
  }
}
@media (max-width: 1000px) {
  .home {
    gap: 4rem;
  }
}
@media(max-width:995px) {
```

```
  .home {
    flex-direction: column;
    margin: 5rem 4rem;
  }
  .home .home-content h3 {
    font-size: 2.5rem;
  }
  .home-content h1 {
    font-size: 5rem;
  }
  .home-img img {
    width: 70vw;
    margin-top: 4rem;
  }
}
```

OUTPUT:



RESULT:

2. JavaScript Form Validation

SOURCE CODE:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Form Validation</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="form-container">
    <h2>Registration Form</h2>
    <form id="myForm">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name">
        <div id="name-error" class="error"></div>

        <label for="email">Email:</label>
        <input type="text" id="email" name="email">
        <div id="email-error" class="error"></div>

        <label for="phone">Phone:</label>
        <input type="text" id="phone" name="phone">
        <div id="phone-error" class="error"></div>

        <label for="password">Password:</label>
        <input type="password" id="password" name="password">
        <div id="password-error" class="error"></div>

        <label for="confirm-password">Confirm Password:</label>
        <input type="password" id="confirm-password" name="confirm-password">
        <div id="confirm-password-error" class="error"></div>

        <button type="submit">Register</button>
    </form>
</div>

<script src="script.js">

</script>

</body>
</html>
```

CSS

```
body {
    font-family: Arial, sans-serif;
    background: #f4f7f8;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
}
.form-container {
    background: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0px 4px 8px rgba(0,0,0,0.1);
    width: 100%;
    max-width: 400px;
}
h2 {
    text-align: center;
    margin-bottom: 20px;
}
label {
    font-weight: bold;
    margin-top: 10px;
    display: block;
}
input {
    width: 100%;
    padding: 10px;
    margin-top: 5px;
    border: 1px solid #ccc;
    border-radius: 5px;
}
.error {
    color: red;
    font-size: 0.9em;
    margin-top: 3px;
}
button {
    background: #4CAF50;
    color: white;
    padding: 10px;
    width: 100%;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    margin-top: 15px;
    cursor: pointer;
```

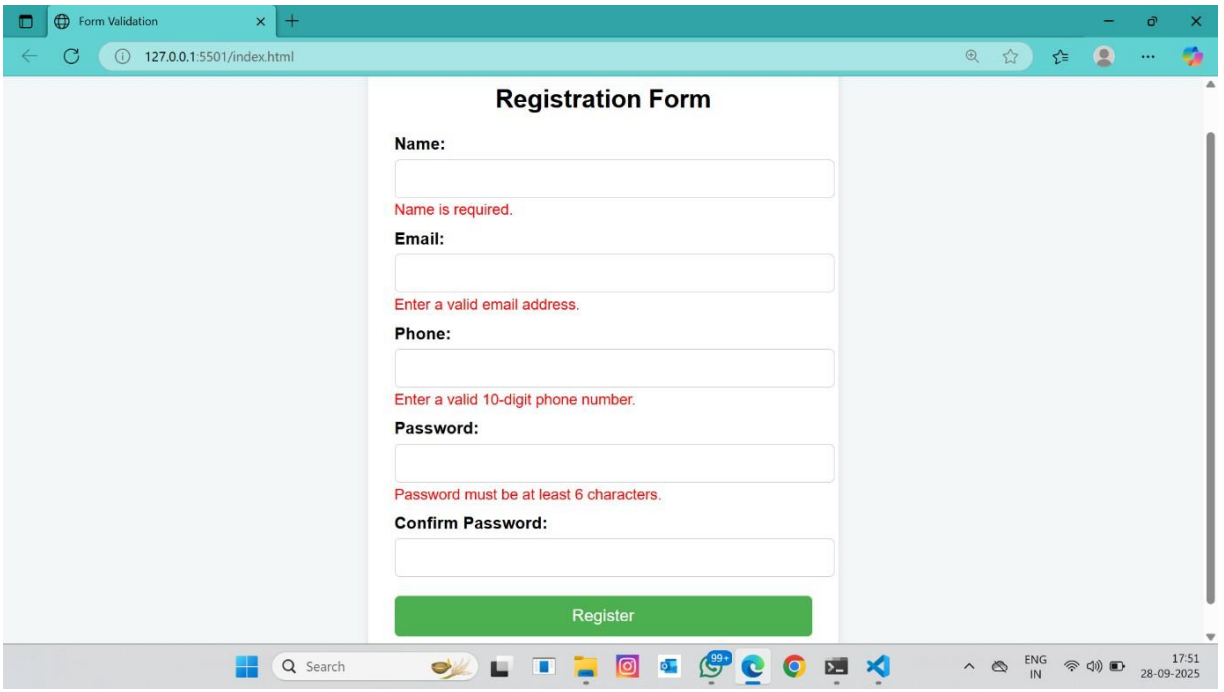
```
}
button:hover {
    background: #45a049;
}
```

SCRIPT

```
const form = document.getElementById("myForm");

form.addEventListener("submit", function(event) {
    event.preventDefault();
    let isValid = true;

    // Get input values
    const name = document.getElementById("name").value.trim();
    const email = document.getElementById("email").value.trim();
    const phone = document.getElementById("phone").value.trim();
    const password = document.getElementById("password").value.trim();
    const confirmPassword = document.getElementById("confirm-password").value.trim();

    // Error elements
    const nameError = document.getElementById("name-error");
    const emailError = document.getElementById("email-error");
    const phoneError = document.getElementById("phone-error");
    const passwordError = document.getElementById("password-error");
    const confirmPasswordError = document.getElementById("confirm-password-error");

    // Clear errors
    nameError.textContent = "";
    emailError.textContent = "";
    phoneError.textContent = "";
    passwordError.textContent = "";
    confirmPasswordError.textContent = "";

    // Name validation
    if (name === "") {
        nameError.textContent = "Name is required.";
        isValid = false;
    }

    // Email validation
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailPattern.test(email)) {
        emailError.textContent = "Enter a valid email address.";
        isValid = false;
    }

    // Phone validation (10 digits)
    const phonePattern = /^[0-9]{10}$/;
```

```
    if (!phonePattern.test(phone)) {
        phoneError.textContent = "Enter a valid 10-digit phone number.";
        isValid = false;
    }

    // Password validation
    if (password.length < 6) {
        passwordError.textContent = "Password must be at least 6 characters.";
        isValid = false;
    }

    // Confirm password
    if (password !== confirmPassword) {
        confirmPasswordError.textContent = "Passwords do not match.";
        isValid = false;
    }

    // If all fields are valid
    if (isValid) {
        alert("Form submitted successfully!");
        form.reset();
    }
});
```

OUTPUT:



RESULT:

3. PHP Contact Form

SOURCE CODE:

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Contact Form</title>
</head>
<body>

    <h2>Contact Us</h2>
    <form action="submit.php" method="POST">
        <label for="name">Name:</label><br>
        <input type="text" id="name" name="name" required><br><br>

        <label for="email">Email:</label><br>
        <input type="email" id="email" name="email" required><br><br>

        <label for="message">Message:</label><br>
        <textarea id="message" name="message" rows="5" required></textarea><br><br>

        <input type="submit" value="Submit">
    </form>

</body>
</html>
```

PHP

```
<?php
// Load Composer's autoloader for PHPMailer
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;

// This path assumes you installed PHPMailer via Composer in the parent directory
require 'vendor/autoload.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // 1. Sanitize and retrieve form data
    $name = htmlspecialchars($_POST['name']);
    $email = htmlspecialchars($_POST['email']);
    $message = htmlspecialchars($_POST['message']);

    // 2. Initialize PHPMailer
    $mail = new PHPMailer(true);

    try {
        // Server settings (Use your Mailtrap/SMTP details here)
```

```
            $mail->isSMTP();
            $mail->Host       = 'sandbox.smtp.mailtrap.io';    // Replace with your SMTP Host (e.g.,
smtp.mailtrap.io)
            $mail->SMTPAuth   = true;
            $mail->Username   = '625da34704a5b1'; // Replace with your SMTP Username
            $mail->Password   = '4c404978968f54'; // Replace with your SMTP Password
            $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
            $mail->Port       = 2525; // Replace with your SMTP Port (e.g., 2525, 587)

            // 3. Recipients
            $mail->setFrom($email, $name);
            $mail->addAddress('nitrajapndi2019@gmail.com', 'Admin'); // The address where you want to
receive the email

            // 4. Content
            $mail->isHTML(true);
            $mail->Subject = "New Contact Form Submission from " . $name;
            $mail->Body    = "
               <h2>New Message Received</h2>
               <p><strong>Name:</strong> {$name}</p>
               <p><strong>Email:</strong> {$email}</p>
               <p><strong>Message:</strong></p>
               <p>" . nl2br($message) . "</p>";
            $mail->AltBody = "Name: {$name}\nEmail: {$email}\nMessage: {$message}";

            // 5. Send
            $mail->send();
            echo "☑ Thank you! Your message has been sent successfully.";

         } catch (Exception $e) {
            // Display error if the email failed to send
            echo "❌ Oops! Something went wrong. Mailer Error: {$mail->ErrorInfo}";
         }
      } else {
         echo "Invalid request method.";
      }
      ?>
```

JSON

```
      {
         "require": {
            "phpmailer/phpmailer": "^6.10"
         }
      }
```

LOCK

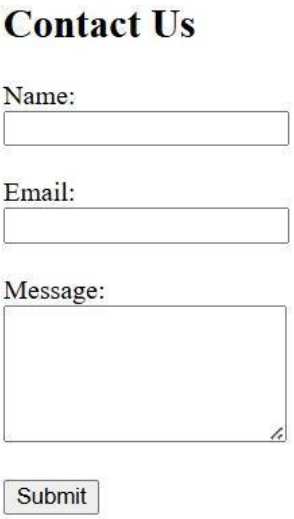

```
{
    "_readme": [
        "This file locks the dependencies of your project to a known state",
        "Read more about it at https://getcomposer.org/doc/01-basic-usage.md#installing-dependencies",
        "This file is @generated automatically"
    ],
    "content-hash": "cb5da62d73716ac8a1776f19367bc28f",
    "packages": [
        {
            "name": "phpmailer/phpmailer",
            "version": "v6.10.0",
            "source": {
                "type": "git",
                "url": "https://github.com/PHPMailer/PHPMailer.git",
                "reference": "bf74d75a1fde6beaa34a0ddae2ec5fce0f72a144"
            },
            "dist": {
                "type": "zip",
                "url":
"https://api.github.com/repos/PHPMailer/PHPMailer/zipball/bf74d75a1fde6beaa34a0ddae2ec5fce0f72a144",
                "reference": "bf74d75a1fde6beaa34a0ddae2ec5fce0f72a144",
                "shasum": ""
            },
            "require": {
                "ext-ctype": "*",
                "ext-filter": "*",
                "ext-hash": "*",
                "php": ">=5.5.0"
            },
            "require-dev": {
                "dealerdirect/phpcodesniffer-composer-installer": "^1.0",
                "doctrine/annotations": "^1.2.6 || ^1.13.3",
                "php-parallel-lint/php-console-highlighter": "^1.0.0",
                "php-parallel-lint/php-parallel-lint": "^1.3.2",
                "phpcompatibility/php-compatibility": "^9.3.5",
                "roave/security-advisories": "dev-latest",
                "squizlabs/php_codesniffer": "^3.7.2",
                "yoast/phpunit-polyfills": "^1.0.4"
            },
            "suggest": {
                "decomplexity/SendOauth2": "Adapter for using XOAUTH2 authentication",
                "ext-mbstring": "Needed to send email in multibyte encoding charset or decode encoded
addresses",
                "ext-openssl": "Needed for secure SMTP sending and DKIM signing",
                "greew/oauth2-azure-provider": "Needed for Microsoft Azure XOAUTH2 authentication",
                "hayageek/oauth2-yahoo": "Needed for Yahoo XOAUTH2 authentication",
                "league/oauth2-google": "Needed for Google XOAUTH2 authentication",
```

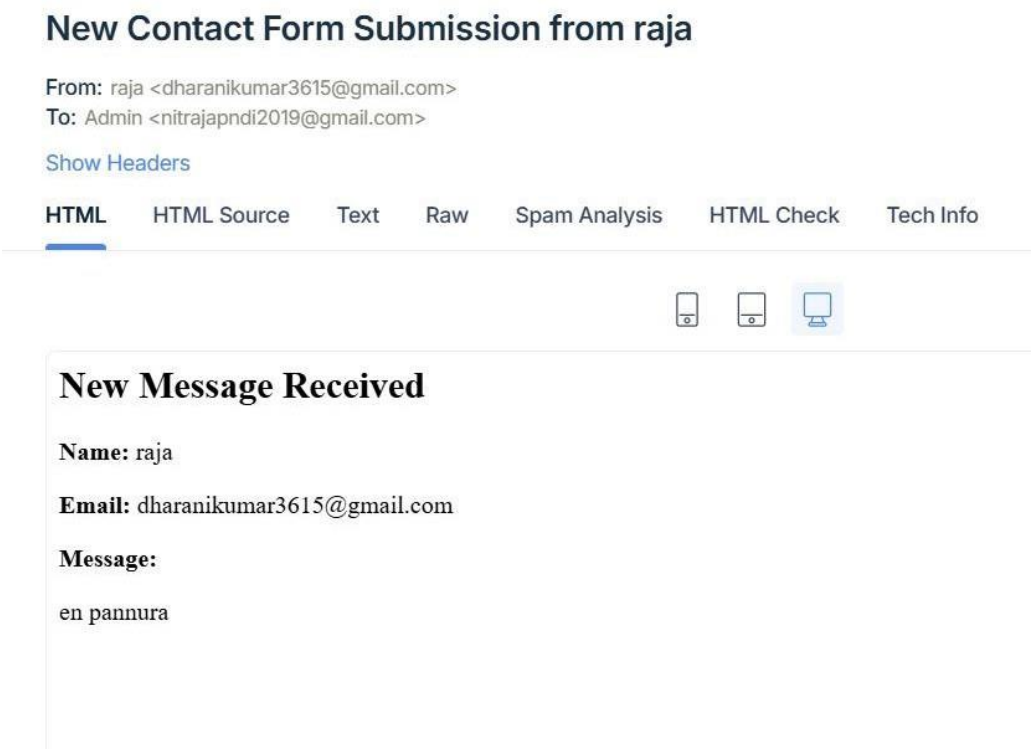
"psr/log": "For optional PSR-3 debug logging",
"symfony/polyfill-mbstring": "To support UTF-8 if the Mbstring PHP extension is not enabled (^1.2)",
"thenetworg/oauth2-azure": "Needed for Microsoft XOAUTH2 authentication"
},

"type": "library",
"autoload": {
"psr-4": {
"PHPMailer\\PHPMailer\\": "src/"
}
},
"notification-url": "https://packagist.org/downloads/",
"license": [
"LGPL-2.1-only"
],
"authors": [
{
"name": "Marcus Bointon",
"email": "phpmailer@synchromedia.co.uk"
},
{
"name": "Jim Jagielski",
"email": "jimjag@gmail.com"
},
{
"name": "Andy Prevost",
"email": "codeworxtech@users.sourceforge.net"
},
{
"name": "Brent R. Matzelle"
}
],
"description": "PHPMailer is a full-featured email creation and transfer class for PHP",
"support": {
"issues": "https://github.com/PHPMailer/PHPMailer/issues",
"source": "https://github.com/PHPMailer/PHPMailer/tree/v6.10.0"
},
"funding": [
{
"url": "https://github.com/Synchro",
"type": "github"
}
],
"time": "2025-04-24T15:19:31+00:00"
}
],
"packages-dev": [],
"aliases": [],
"minimum-stability": "stable",

"stability-flags": {},
"prefer-stable": false,
"prefer-lowest": false,
"platform": {},
"platform-dev": {},
"plugin-api-version": "2.6.0"
}

OUTPUT:





RESULT:

4.SQL CRUD Operations

SOURCE CODE:

INDEX.PHP

```
<?php include 'db.php';

// CREATE logic
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST["name"];
    $email = $_POST["email"];
    $conn->query("INSERT INTO users (name, email) VALUES ('$name', '$email')");
    header("Location: index.php");
}
$users = $conn->query("SELECT * FROM users");
?>

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CRUD with PHP & Tailwind</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 p-8">
  <div class="max-w-2xl mx-auto bg-white p-6 rounded shadow">
    <h2 class="text-2xl font-bold mb-4">Add User</h2>
    <form method="POST" class="space-y-4">
      <input name="name" placeholder="Name" class="w-full p-2 border rounded" required>
      <input name="email" placeholder="Email" class="w-full p-2 border rounded" required>
      <button class="bg-blue-500 text-white px-4 py-2 rounded">Submit</button>
    </form>
  </div>

  <div class="max-w-2xl mx-auto mt-8 bg-white p-6 rounded shadow">
    <h2 class="text-xl font-bold mb-4">User List</h2>
    <table class="w-full text-left">
      <thead>
        <tr>
          <th class="border-b p-2">#</th>
          <th class="border-b p-2">Name</th>
          <th class="border-b p-2">Email</th>
          <th class="border-b p-2">Actions</th>
        </tr>
      </thead>
      <tbody>
        <?php while ($row = $users->fetch_assoc()): ?>
        <tr>
          <td class="border-b p-2"><?= $row['id'] ?></td>
          <td class="border-b p-2"><?= $row['name'] ?></td>
```

```
            <td class="border-b p-2"><?= $row['email'] ?></td>
            <td class="border-b p-2 space-x-2">
              <a href="edit.php?id=<?= $row['id'] ?>" class="text-blue-600">Edit</a>
              <a href="delete.php?id=<?= $row['id'] ?>" onclick="return confirm('Delete this user?')"
class="text-red-600">Delete</a>
            </td>
          </tr>
          <?php endwhile; ?>
        </tbody>
      </table>
    </div>
  </body>
  </html>
```

```
CREATE DATABASE crud_app;
USE crud_app;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);
```

DB.PHP

```
<?php
$conn = new mysqli("localhost", "root", "", "crud_db");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

EDIT.PHP

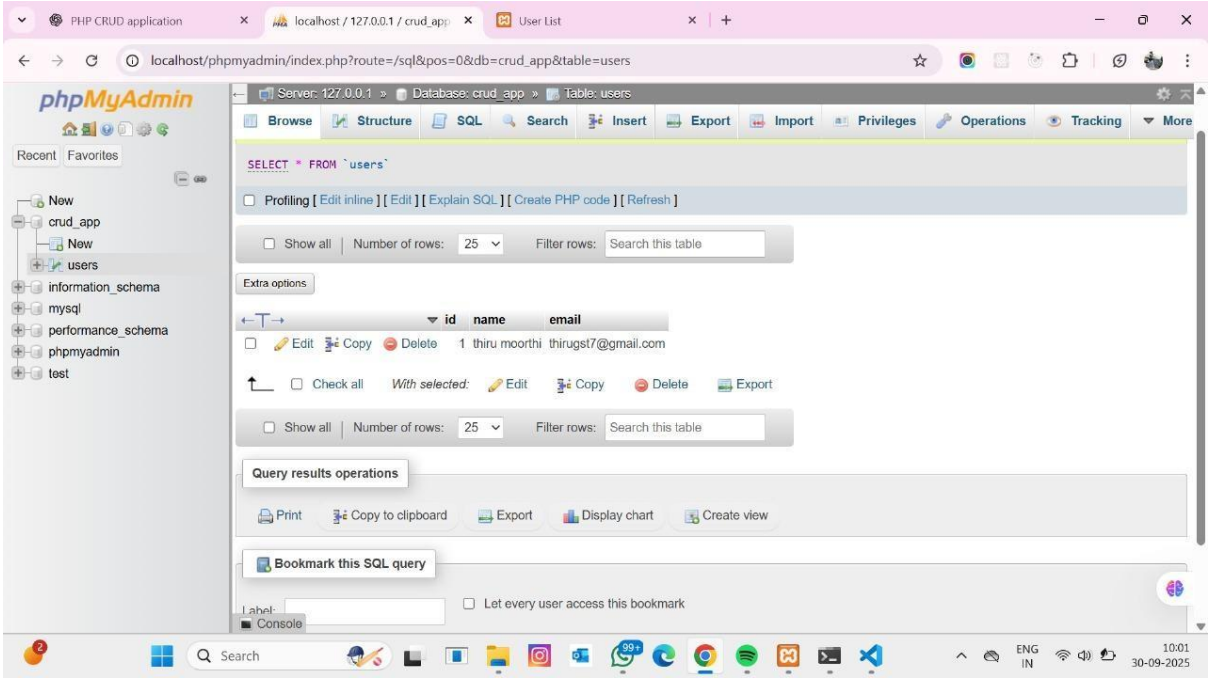
```
<?php
include 'db.php';
$id = $_GET['id'];

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST["name"];
    $email = $_POST["email"];
    $conn->query("UPDATE users SET name='$name', email='$email' WHERE id=$id");
    header("Location: index.php");
}

$result = $conn->query("SELECT * FROM users WHERE id=$id");
$user = $result->fetch_assoc();
?>
```

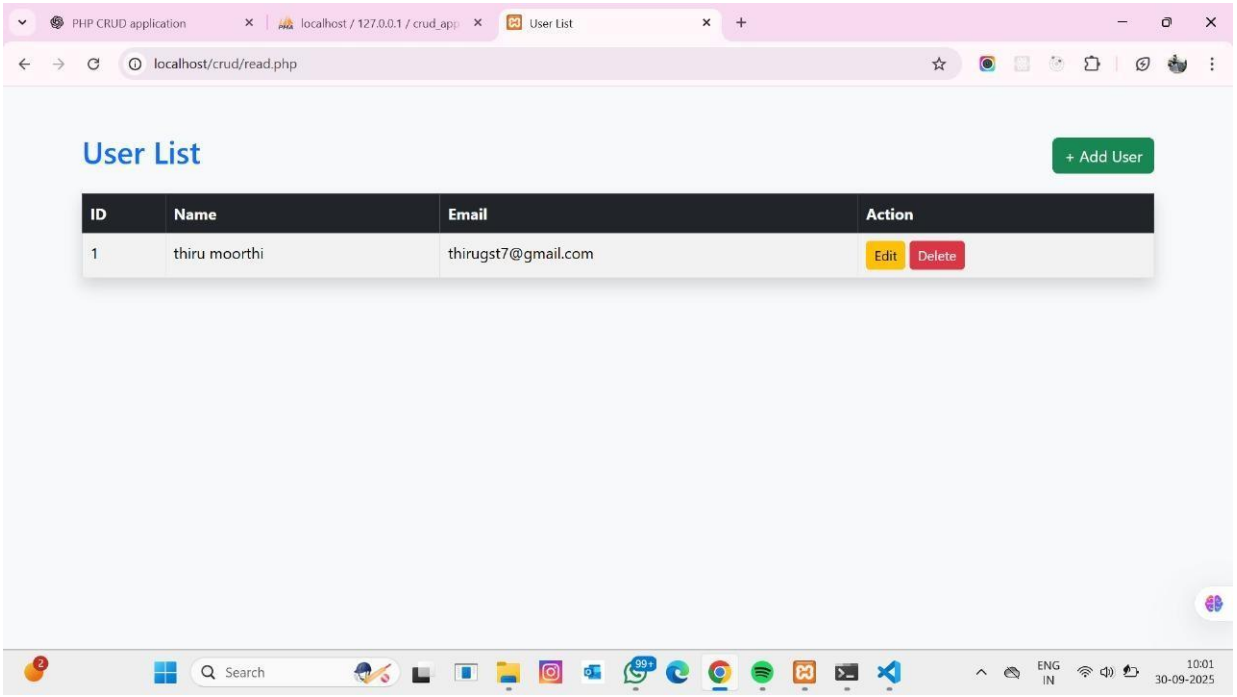
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Edit User</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 p-8">
  <div class="max-w-xl mx-auto bg-white p-6 rounded shadow">
    <h2 class="text-2xl font-bold mb-4">Edit User</h2>
    <form method="POST" class="space-y-4">
      <input name="name" value="<?= $user['name'] ?>" class="w-full p-2 border rounded" required>
      <input name="email" value="<?= $user['email'] ?>" class="w-full p-2 border rounded" required>
      <button class="bg-green-500 text-white px-4 py-2 rounded">Update</button>
      <a href="index.php" class="ml-4 text-blue-600">Cancel</a>
    </form>
  </div>
</body>
</html>
```

DELETE.PHP

```
<?php
include 'db.php';
$id = $_GET['id'];
$conn->query("DELETE FROM users WHERE id=$id");
header("Location: index.php");
?>
```

OUTPUT:





RESULT:

5. React Components - State & Props

SORCE CODE

App.jsx:

```
import React, { useState } from "react";
import "./App.css";

function Counter({ initialCount }) {
  const [count, setCount] = useState(initialCount);
  const [animate, setAnimate] = useState(false);

  const handleClick = (value) => {
    setCount(count + value);
    setAnimate(true);
    setTimeout(() => setAnimate(false), 300); // reset animation
  };

  return (
    <div className="counter-card">
      <h2 className={`count ${animate ? "animate" : ""}`}>{count}</h2>
      <div className="buttons">
        <button onClick={() => handleClick(1)}>Increment</button>
        <button onClick={() => handleClick(-1)}>Decrement</button>
        <button onClick={() => setCount(initialCount)}>Reset</button>
      </div>
    </div>
  );
}

function App() {
  return (
    <div className="app-container">
      <h1>Props + State Component</h1>
```

```
<Counter initialCount={0} />
```

```
    </div>
  );
}

export default App;
```

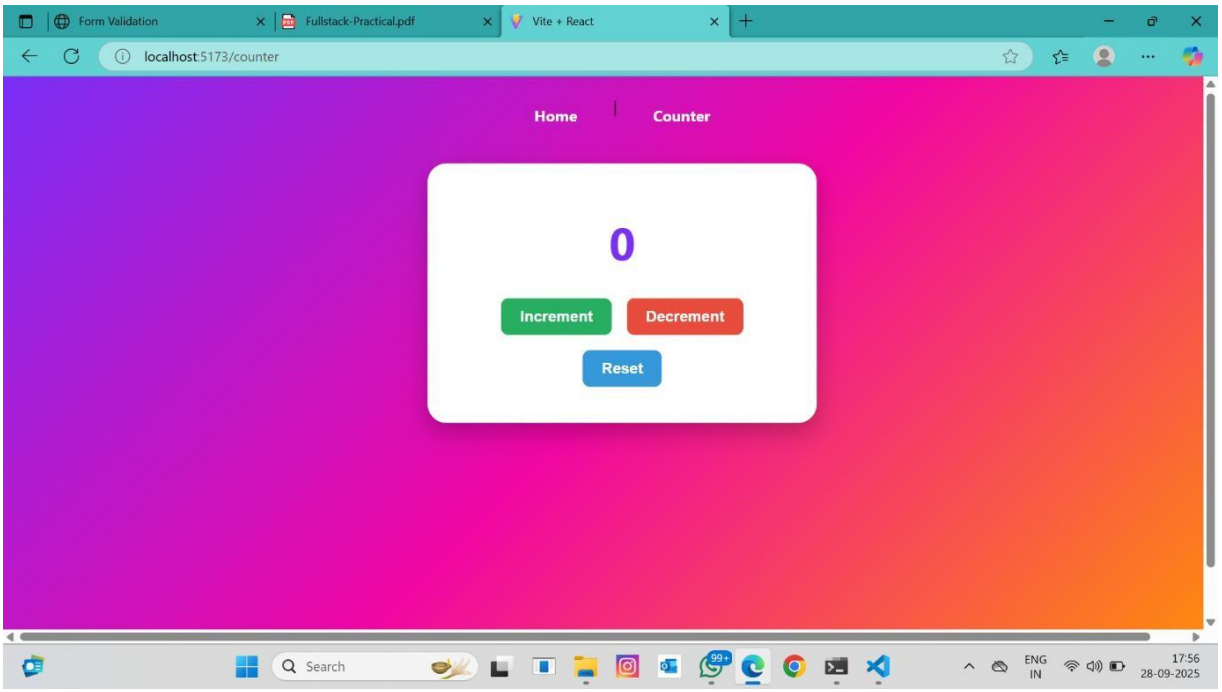
App.css:

```
/* Ensure background covers full page */
html, body, #root {
  height: 100%;
  margin: 0;
  color:red;
}

/* Full-page centered layout */
.app-container {
  height:  100vh;
  width: 100%;
  background: linear-gradient(135deg, #7b2ff7, #f107a3, #ff9800);
  display: flex;
  flex-direction: column;
  justify-content: center;  /* Vertical center */
  align-items: center;      /* Horizontal center */
  text-align: center;
}

.app-container h1 {
  color: white;
  margin-bottom: 20px;
  font-family: "Segoe UI", sans-serif;
  text-shadow: 2px 2px 6px rgba(0,0,0,0.4);
}
```

```
/* Counter Card */
.counter-card {
  background: white;
  padding: 30px;
  border-radius: 20px;
  box-shadow: 0 10px 25px rgba(0,0,0,0.25);
  text-align: center;
  max-width: 350px;
}

/* Count Display */
.count {
  font-size: 3rem;
  font-weight: bold;
  color: #7b2ff7;
  margin: 20px 0;
  transition: transform 0.3s ease, color 0.3s ease;
}
.count.animate {
  transform: scale(1.3);
  color: #f107a3;
}

/* Buttons */
.buttons button {
  padding: 10px 20px;
  margin: 8px;
  font-size: 1rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
```

```
  color: #fff;
  font-weight: bold;
  transition: opacity 0.3s ease;
}
.buttons button:nth-child(1) {
  background: #27ae60; /* Green */
}
.buttons button:nth-child(2) {
  background: #e74c3c; /* Red */
}
.buttons button:nth-child(3) {
  background: #3498db; /* Blue */
}
.buttons button:hover {
  opacity: 0.8;
}
```

OUTPUT:



RESULT:

6.Single Page Application with React Router

SORCE CODE:

App.jsx:

```
import React, { useState } from "react";
import "./App.css";

function App() {
  const [task, setTask] = useState("");
  const [tasks, setTasks] = useState([]);
  const [editId, setEditId] = useState(null);

  // Add or Update Task
  const handleTask = () => {
    if (task.trim() === "") return;

    if (editId) {
      setTasks(tasks.map((t) => (t.id === editId ? { ...t, text: task } : t)));
      setEditId(null);
      setTask("");
    } else {
      setTasks([...tasks, { text: task, id: Date.now(), done: false }]);
      setTask("");
    }
  };

  // Delete Task
  const deleteTask = (id) => {
    // Add 'removing' class for CSS animation
    const element = document.getElementById(`task-${id}`);
    element.classList.add("remove");
    setTimeout(() => {
      setTasks(tasks.filter((t) => t.id !== id));
    }, 300); // match with CSS animation duration
```

```
};

// Edit Task
const editTask = (id) => {
  const taskToEdit = tasks.find((t) => t.id === id);
  setTask(taskToEdit.text);
  setEditId(id);
};

// Toggle Completed
const toggleDone = (id) => {
  setTasks(
    tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t))
  );
};

return (
  <div className="container">
    <div className="todo-card">
      <h1> ✨ To-Do List ✨ </h1>

      <input
        type="text"
        value={task}
        onChange={(e) => setTask(e.target.value)}
        placeholder="Enter a task..."
      />

      <button onClick={handleTask}>{editId ? "Update" : "Add"}</button>

      <ul>
        {tasks.map((t) => (
```

```
          <li
            key={t.id}
            id={`task-${t.id}`}
            className={`task-item ${t.done ? "done" : ""} add`}
          >
            <span onClick={() => toggleDone(t.id)}>{t.text}</span>
            <div className="buttons">
              <button className="edit-btn" onClick={() => editTask(t.id)}>
                Edit
              </button>
              <button className="delete-btn" onClick={() => deleteTask(t.id)}>
                Delete
              </button>
            </div>
          </li>
        ))}
      </ul>
    </div>
  </div>
);
}

export default App;
```

App.css:

```
/* General page */
body, html {
margin: 0;
  padding: 0;
  font-family: 'Arial', sans-serif;
  background: linear-gradient(to right, #7b2ff7, #f107a3);
  min-height: 100vh;
  display: flex;
```

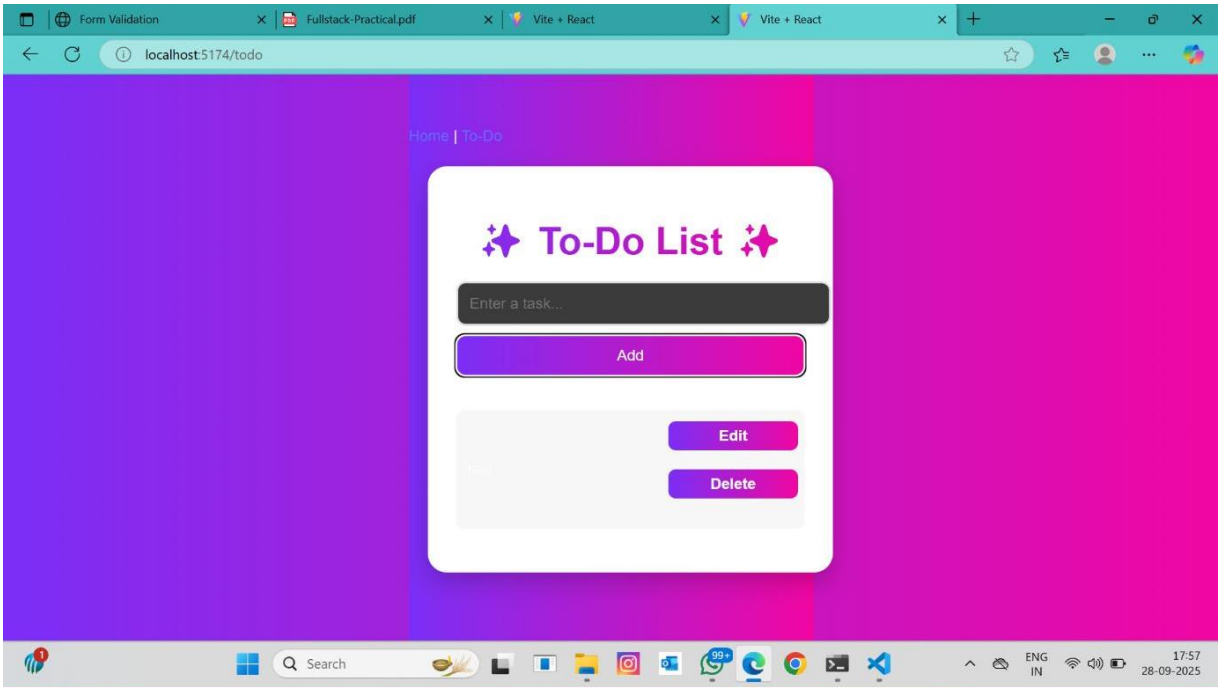
```
  justify-content: center;
  align-items: center;
}

/* Container for To-Do */
.container {
  width: 100%;
  max-width: 500px;
  padding: 20px;
}

/* Card style */
.todo-card {
  background-color: #fff;
  border-radius: 20px;
  padding: 30px;
  box-shadow: 0 10px 25px rgba(0,0,0,0.2);
}

/* Heading */
.todo-card h1 {
  text-align: center;
  font-size: 2.5rem;
  background: linear-gradient(to right, #7b2ff7, #f107a3);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  margin-bottom: 20px;
}

/* Input */
.todo-card input {
  width: 100%;
```

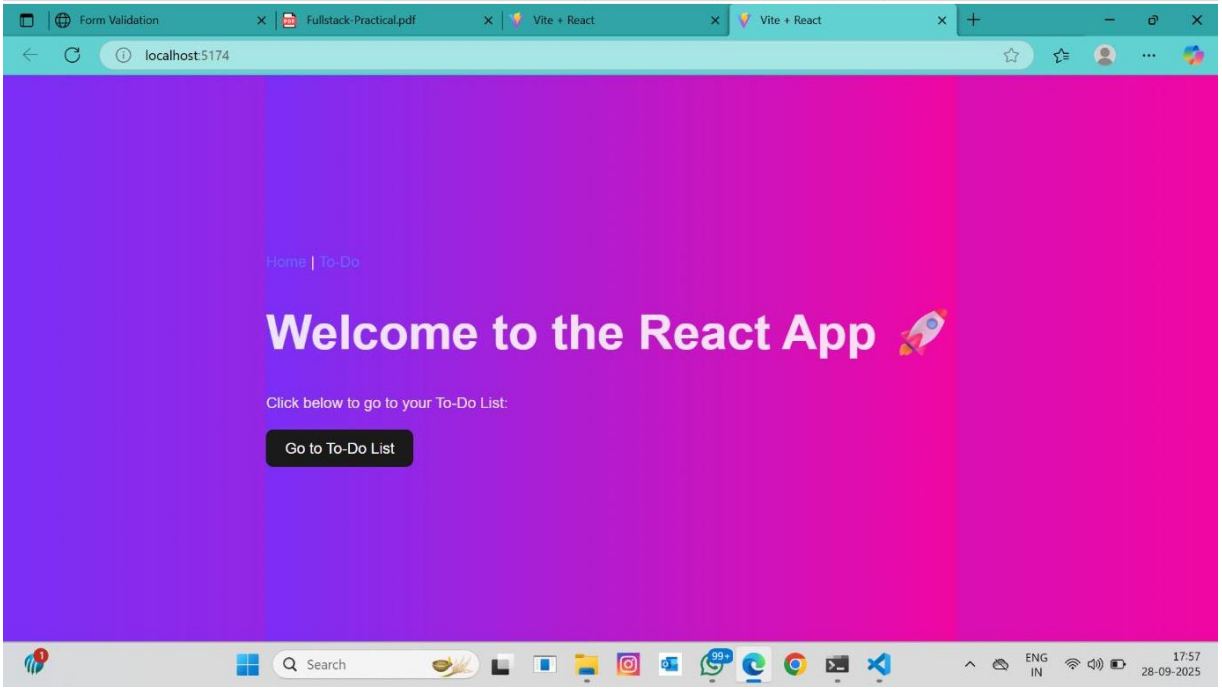
```
  padding: 12px;
  border-radius: 10px;
  border: 2px solid #ddd;
  margin-bottom: 10px;
  font-size: 1rem;
}

/* Add / Update Button */
.todo-card button {
  width: 100%;
  padding: 12px;
  background: linear-gradient(to right, #7b2ff7, #f107a3);
  color: #fff;
  border: none;
  border-radius: 10px;
  font-size: 1rem;
  cursor: pointer;
  margin-bottom: 20px;
  transition: opacity 0.3s ease;
}
.todo-card button:hover {
  opacity: 0.8;
}

/* Task List */
.todo-card ul {
  list-style: none;
  padding: 0;
}

/* Task Item */
.task-item {
```

```
  background: #f7f7f7;
  padding: 12px;
  margin-bottom: 10px;
  border-radius: 10px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: transform 0.3s ease, opacity 0.3s ease;
  animation: fadeIn 0.3s ease forwards;
  cursor: pointer;
}

.task-item.done span {
  text-decoration: line-through;
  color: gray;
}

/* Fade-in animation */
@keyframes fadeIn {
  from {opacity: 0; transform: translateY(-20px);}
  to {opacity: 1; transform: translateY(0);}
}

/* Remove animation */
.task-item.remove {
  animation: fadeOut 0.3s ease forwards;
}

@keyframes fadeOut {
  from {opacity: 1; transform: translateX(0);}
  to {opacity: 0; transform: translateX(50px);}
}
```

```
/* Buttons inside task */
.buttons button {
  padding: 6px 12px;
  margin-left: 5px;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  font-weight: bold;
  transition: opacity 0.3s ease;
}
.edit-btn {
  background-color: #3498db;
  color: #fff;
}
.edit-btn:hover {
  opacity: 0.8;
}
.delete-btn {
  background-color: #e74c3c;
  color: #fff;
}
.delete-btn:hover {
  opacity: 0.8;
}
```

OUTPUT:





RESULT:

7.Create RESTful API with Node.js & Express

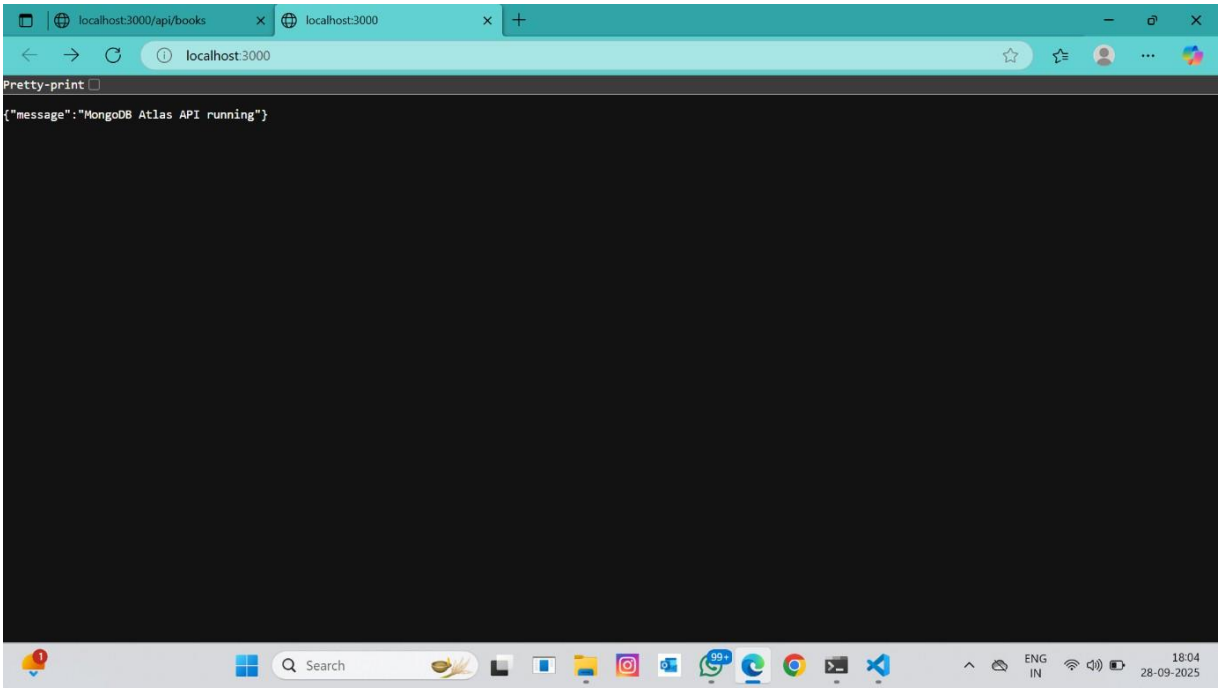
SORCE CODE:

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
require("dotenv").config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middlewares
app.use(cors());
app.use(express.json());

// Connect to MongoDB Atlas
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB Atlas connected"))
  .catch((err) => console.error("MongoDB connection error:", err));

// Book schema
const bookSchema = new mongoose.Schema({
  title: String,
  author: String,
  year: Number,
});

const Book = mongoose.model("Book", bookSchema);

// Routes
app.get("/", (req, res) => {
  res.json({ message: "MongoDB Atlas API running" });
});
```

```
// ☑ Get all books
app.get("/api/books", async (req, res) => {
  try {
    const books = await Book.find();
    res.json(books);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ☑ Add book
app.post("/api/books", async (req, res) => {
  try {
    const book = new Book(req.body);
    await book.save();
    res.status(201).json(book);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ☑ Update book
app.put("/api/books/:id", async (req, res) => {
  try {
    const updatedBook = await Book.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true } // return updated doc
    );
    res.json(updatedBook);
  } catch (err) {
```

```
    res.status(500).json({ error: err.message });
  }
});

// ☑ Delete book
app.delete("/api/books/:id", async (req, res) => {
  try {
    await Book.findByIdAndDelete(req.params.id);
    res.json({ message: "Book deleted" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Start server
app.listen(PORT, () =>
  console.log(`Server running on http://localhost:${PORT}`)
);
```

OUTPUT:



RESULT:

8. MongoDB CRUD Integration with Mongoose

SORCE CODE:

Steps:

Install MongoDB

Login with gmail

Open Mongo Atlas

Create cluster

Copy cluster string

Integration with Mongo DB

Connect the Mongo DB and Atlas

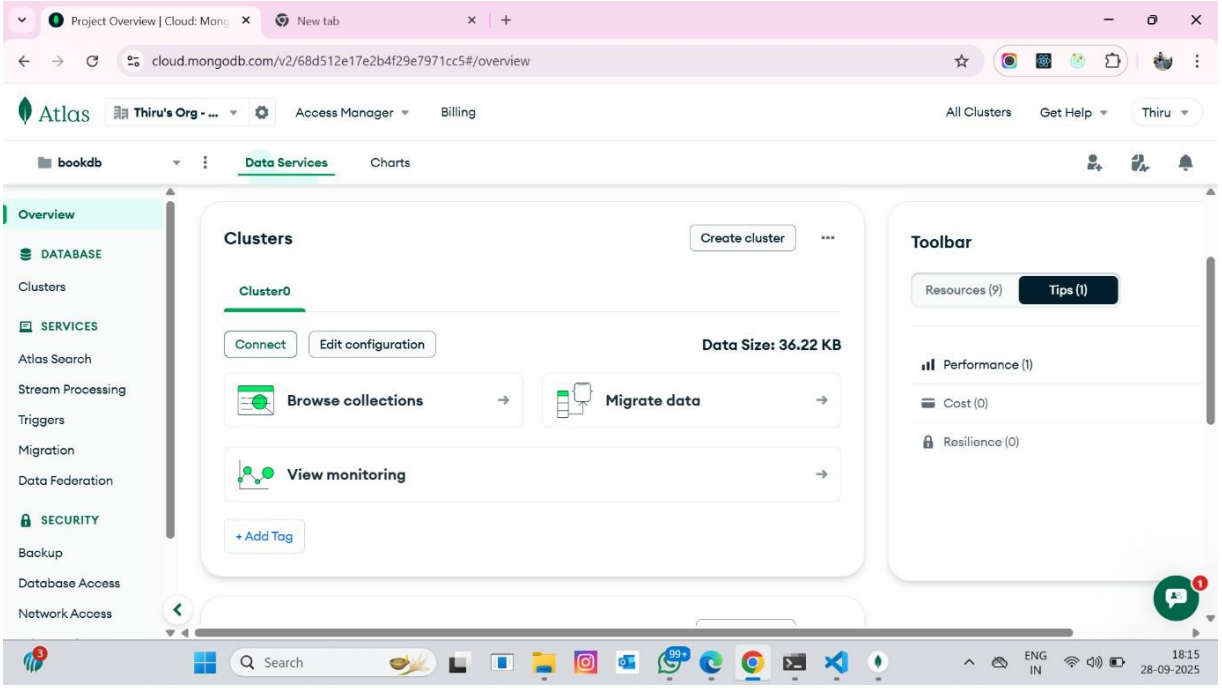
Copy the connect string and paste in .env

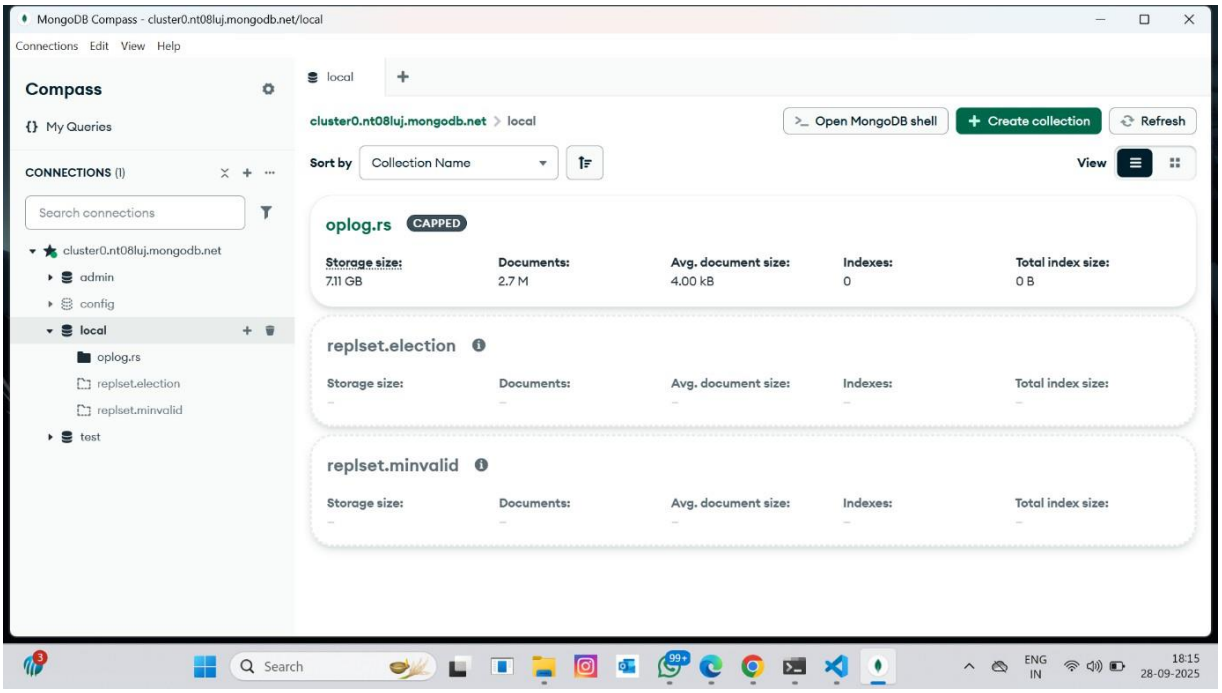
```
/ Connect to MongoDB Atlas
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB Atlas connected"))
  .catch((err) => console.error("MongoDB connection error:", err));
```

.env:

```
PORT=3000
MONGO_URI=mongodb+srv://thirugst7_db_user:Thiru1432@cluster0.nt08luj.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0
```

OUTPUT:





RESULT:

9. Authentication using JWT & bcrypt

SOURCE CODE:

SERVER:

```
require('dotenv').config();
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

const app = express();
app.use(express.json());

// In-memory user store (replace with DB in production)
const users = [];

// config helpers
const SALT_ROUNDS = parseInt(process.env.BCRYPT_SALT_ROUNDS, 10) || 12;
const JWT_SECRET = process.env.JWT_SECRET;
const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || '1h';
const PORT = process.env.PORT || 3000;

if (!JWT_SECRET) {
  console.error('Missing JWT_SECRET in .env — exit.');
  process.exit(1);
}

// Register endpoint
app.post('/register', async (req, res) => {
  try {
    const { username, password } = req.body;
    if (!username || !password) return res.status(400).json({ message: 'username & password required' });

    const exists = users.find(u => u.username === username);
```

```
    if (exists) return res.status(409).json({ message: 'username already taken' });

    const hash = await bcrypt.hash(password, SALT_ROUNDS);
    const user = { id: users.length + 1, username, passwordHash: hash };
    users.push(user);

    // return minimal user info (never return password/hash)
    res.status(201).json({ id: user.id, username: user.username });
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: 'server error' });
  }
});

// Login endpoint -> issues JWT
app.post('/login', async (req, res) => {
try {
    const { username, password } = req.body;
    if (!username || !password) return res.status(400).json({ message: 'username & password required' });

    const user = users.find(u => u.username === username);
    if (!user) return res.status(401).json({ message: 'invalid credentials' });

    const ok = await bcrypt.compare(password, user.passwordHash);
    if (!ok) return res.status(401).json({ message: 'invalid credentials' });

    // create JWT (HS256). Keep payload minimal.
    const payload = { userId: user.id };
    const token = jwt.sign(payload, JWT_SECRET, { algorithm: 'HS256', expiresIn: JWT_EXPIRES_IN
});
```

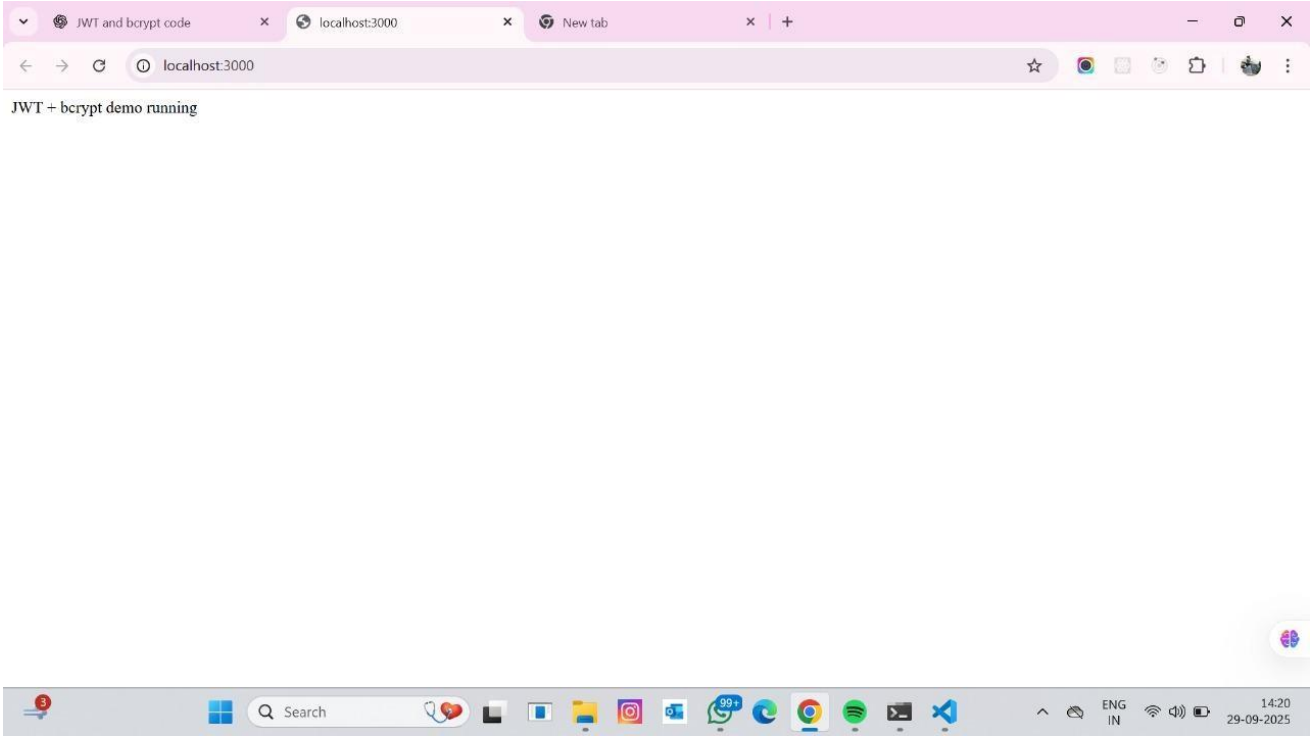
```
    // Return token (client may store it in memory / HttpOnly cookie — see notes)
    res.json({ token });
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: 'server error' });
  }
});

// Middleware to protect routes
function authenticateJWT(req, res, next) {
  const auth = req.headers.authorization;
  if (!auth) return res.status(401).json({ message: 'missing authorization header' });

  const parts = auth.split(' ');
  if (parts.length !== 2 || parts[0] !== 'Bearer') return res.status(401).json({ message: 'malformed token'
});


  const token = parts[1];
  // Verify and allow only specific algorithms for extra safety
  jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] }, (err, payload) => {
    if (err) return res.status(401).json({ message: 'invalid or expired token' });
    req.userId = payload.userId;
    next();
  });
}

// protected route example
app.get('/protected', authenticateJWT, (req, res) => {
  res.json({ message: `Hello user ${req.userId} — this is protected.` });
});
```

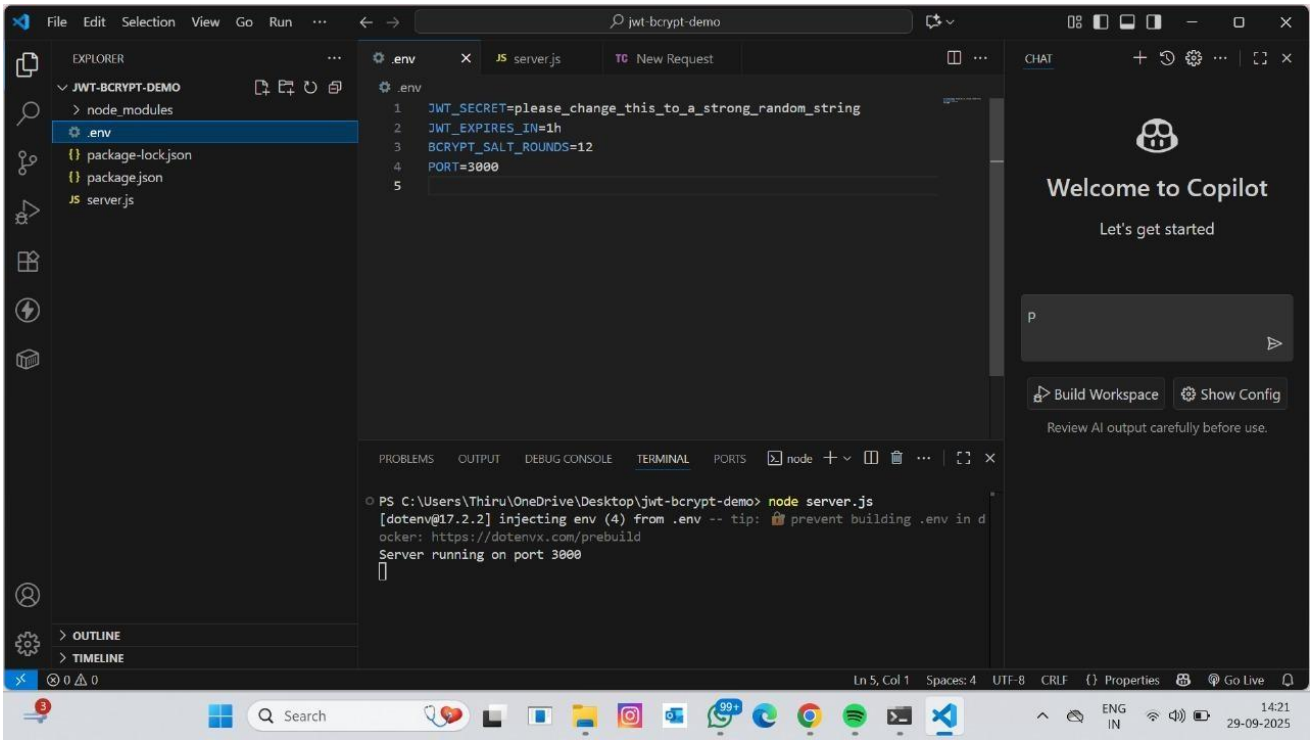
```
// simple status
app.get('/', (req, res) => res.send('JWT + bcrypt demo running'));

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

env:

```
JWT_SECRET="hhhahhhahhhahhahahahjhi8jdjkdnjdbhbsbanmanhabdnchjd"
JWT_EXPIRES_IN=1h
BCRYPT_SALT_ROUNDS=12
PORT=3000
```

OUTPUT:





RESULT:

10. Connect React Frontend with Express API

SOURCE CODE

Booklist.js:

```
import React, { useEffect, useState } from "react";
import axios from "axios";
import "../App.css"; // ☑ Correct import

const API_URL = "http://localhost:3000/api/books";

function BookList() {
  const [books, setBooks] = useState([]);
  const [form, setForm] = useState({ title: "", author: "", year: "" });
  const [editingBook, setEditingBook] = useState(null);
  const [animateId, setAnimateId] = useState(null);

  useEffect(() => {
    fetchBooks();
  }, []);

  const fetchBooks = async () => {
    try {
      const res = await axios.get(API_URL);
      setBooks(res.data);
    } catch (err) {
      console.error(err);
    }
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const bookData = { ...form, year: Number(form.year) };
      let updatedBook;
```

```
    if (editingBook) {
      updatedBook = await axios.put(`${API_URL}/${editingBook._id}`, bookData);
      setEditingBook(null);
    } else {
      updatedBook = await axios.post(API_URL, bookData);
      updatedBook = { data: { ...bookData, _id: Math.random().toString(36) } }; // temp ID for animation
    }

    setAnimateId(updatedBook.data._id);
    setForm({ title: "", author: "", year: "" });
    fetchBooks();
  } catch (err) {
    console.error(err);
  }
};

const handleDelete = async (id) => {
  try {
    await axios.delete(`${API_URL}/${id}`);
    fetchBooks();
  } catch (err) {
    console.error(err);
  }
};

const handleEdit = (book) => {
  setEditingBook(book);
  setForm({ title: book.title, author: book.author, year: book.year });
};

return (
```

```
<div className="container">
  <h1>Movie Manager</h1>

  <form onSubmit={handleSubmit} className="form">
    <input
      type="text"
      placeholder="Title"
      value={form.title}
      onChange={(e) => setForm({ ...form, title: e.target.value })}
      required
    />
    <input
      type="text"
      placeholder="Director"
      value={form.author}
      onChange={(e) => setForm({ ...form, author: e.target.value })}
      required
    />
    <input
      type="number"
      placeholder="Year"
      value={form.year}
      onChange={(e) => setForm({ ...form, year: e.target.value })}
      required
    />
    <button type="submit">{editingBook ? "Update Movie" : "Add Movie"}</button>
  </form>

  <ul className="book-list">
    {books.map((book) => (
      <li
        key={book._id}
```

```
            className={`book-item ${animateId === book._id ? "animate-pop" : ""}`}
          >
            <b>{book.title}</b> by {book.author} ({book.year})
            <div className="actions">
              <button onClick={() => handleEdit(book)}>Edit</button>
              <button onClick={() => handleDelete(book._id)}>Delete</button>
            </div>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default BookList;
```

App.js:

```
import React from "react";
import BookList from "./components/BookList";
import "./index.css";
function App() {
  return (
    <div>
      <BookList />
    </div>
  );
}

export default App;
```

App.css:

```
/* ---------------------
```

```
  Global & Body
---------------------- */
/* --------------------
  Global & Body
---------------------- */
body, html, #root {
  margin: 0;
  padding: 0;
  height: 100%;
  font-family: 'Segoe UI', sans-serif;
  /* Professional 2-color gradient background */
  background: linear-gradient(135deg, #2f3640, #fbc531);
  display: flex;
  justify-content: center;  /* horizontal center */
  align-items: center;      /* vertical center */
  color: #333;
}

/* --------------------
  Container
---------------------- */
.container {
  width: 100%;
  max-width: 700px;
  padding: 20px;
  display: flex;
  flex-direction: column;
  align-items: center; /* center form and list */
}

/* --------------------
  Page Title
```

```
---------------------- */
h1 {
  text-align: center;
  color: #2f3640; /* dark text */
  font-size: 2.8rem;
  margin-bottom: 25px;
}

/* --------------------
   Form Styles
---------------------- */
form {
  background: white;
  padding: 25px;
  border-radius: 15px;
  box-shadow: 0 8px 20px rgba(0,0,0,0.1);
  display: flex;
  flex-direction: column;
  gap: 12px;
  width: 100%;
  max-width: 500px;
  transition: transform 0.3s ease;
}
form:hover {
  transform: scale(1.01);
}

form input {
  padding: 10px 14px;
  border: 1px solid #ccc;
  border-radius: 10px;
  font-size: 1rem;
```

```
  transition: border 0.3s, box-shadow 0.3s;
}
form input:focus {
  outline: none;
  border-color: #2f3640;
  box-shadow: 0 0 5px rgba(47,54,64,0.3);
}

form button {
  padding: 12px;
  border: none;
  border-radius: 12px;
  font-size: 1.1rem;
  font-weight: bold;
  color: white;
  background-color: #2f3640;
  cursor: pointer;
  transition: background 0.3s, transform 0.3s;
}
form button:hover {
  background-color: #353b48;
  transform: scale(1.03);
}

/* ---------------------
   Book List
---------------------- */
.book-list {
  margin-top: 25px;
  width: 100%;
  display: flex;
  flex-direction: column;
```

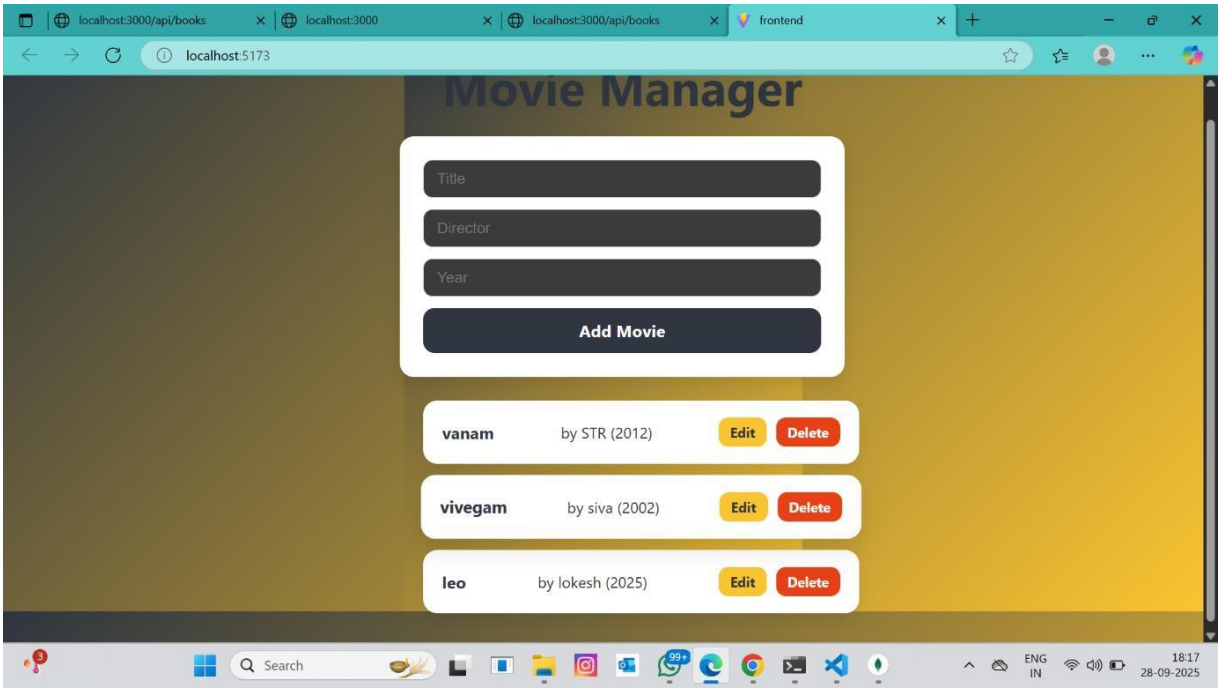
```
  gap: 12px;
  align-items: center; /* center all book cards */
}

.book-item {
  background: white;
  padding: 18px 20px;
  border-radius: 15px;
  box-shadow: 0 6px 18px rgba(0,0,0,0.08);
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
  width: 100%;
  max-width: 500px;
}
.book-item:hover {
  transform: scale(1.01);
  box-shadow: 0 10px 25px rgba(0,0,0,0.12);
}

.book-item b {
  font-size: 1.1rem;
  color: #2f3640;
}

/* ---------------------
   Actions Buttons
---------------------- */
.actions {
  display: flex;
  gap: 10px;
```

```
}

.actions button {
  padding: 6px 12px;
  border: none;
  border-radius: 10px;
  cursor: pointer;
  font-weight: bold;
  font-size: 0.9rem;
  transition: transform 0.2s, background 0.3s;
}

.actions button:hover {
  transform: scale(1.05);
}

/* Edit button */
.actions button:first-child {
  background-color: #fbc531;
  color: #2f3640;
}
.actions button:first-child:hover {
  background-color: #e1b12c;
}

/* Delete button */
.actions button:last-child {
  background-color: #e84118;
  color: white;
}
.actions button:last-child:hover {
  background-color: #c23616;
```

```
}

/* ---------------------
   Animation for new/updated books
---------------------- */
@keyframes pop {
  0% { transform: scale(1); }
  50% { transform: scale(1.08); }
  100% { transform: scale(1); }
}

.animate-pop {
  animation: pop 0.3s ease-in-out;
}

/* ---------------------
   Responsive Design
---------------------- */
/* Mobile (up to 480px) */
@media (max-width: 480px) {
  h1 {
    font-size: 2rem;
  }
  form, .book-item {
    width: 90%;
  }
  .actions {
    flex-wrap: wrap;
    justify-content: flex-start;
  }
}
```

```
/* Tablet (481px - 768px) */
@media (min-width: 481px) and (max-width: 768px) {
  h1 {
    font-size: 2.3rem;
  }
  form, .book-item {
    width: 80%;
  }
}

/* Laptops & Desktops (769px and up) */
@media (min-width: 769px) {
  h1 {
    font-size: 2.8rem;
  }
  form, .book-item {
    width: 100%;
  }
}
```

OUTPUT:



RESULT:

11. Dockerize Full Stack App

SOURCE CODE:

Dockerfile for backend:

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Dockerfile for front end:

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev"]
```

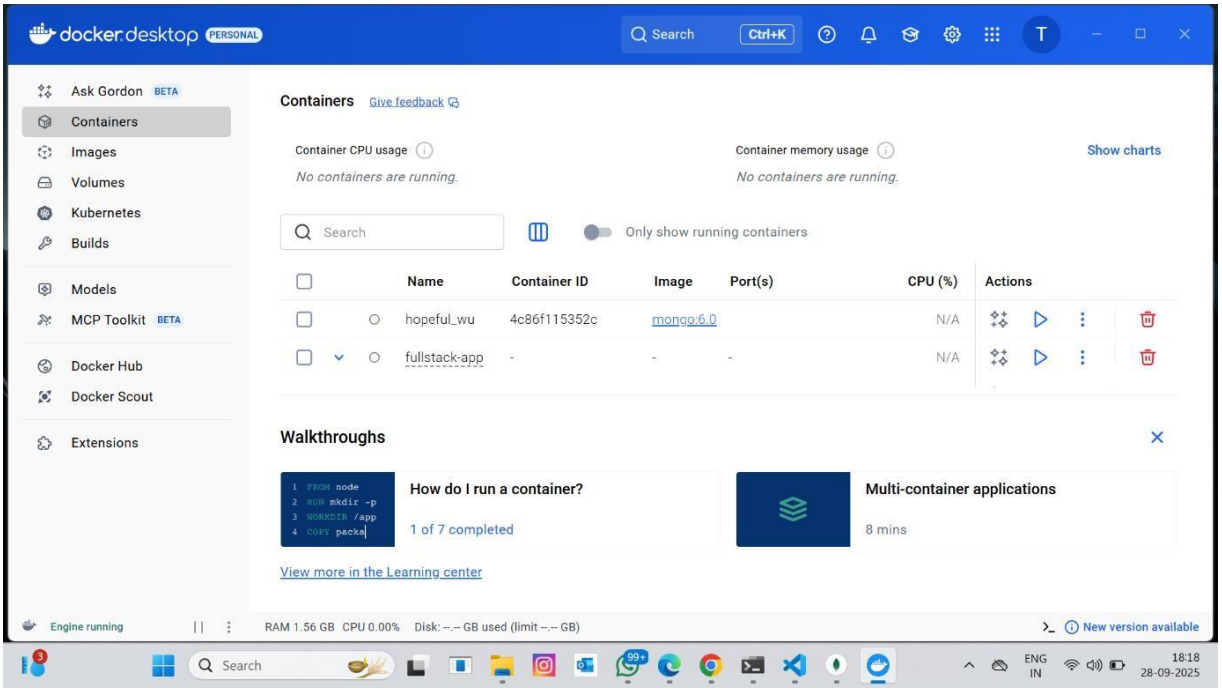
Docker-compose.yml:

```
services:
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend
    environment:
      - VITE_API_URL=http://localhost:3000

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    depends_on:
      - mongo
    environment:
      - MONGO_URI=mongodb://mongo:27017/booksdb

  mongo:
    image: mongo
    restart: always
    ports:
      - "27017:27017"
```

.Dockerignore:

```
node_modules
npm-debug.log
dist
.env
```

OUTPUT:



RESULT:

12. Automate CI/CD with GitHub Actions

SOURCE CODE:

name: CI — frontend & backend tests

```
on:
  pull_request:
  push:
    branches: [ develop, feature/** ]

jobs:
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: node-version: 20
      - name: Install deps
        working-directory: frontend
        run: npm ci
      - name: Lint
        working-directory: frontend
        run: npm run lint --if-present
      - name: Tests
        working-directory: frontend
        run: npm test --if-present -----watchAll=false

  backend:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: postgres
        POSTGRES_PASSWORD: postgres
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: node-version: 20
      - name: Install deps
        working-directory: backend
        run: npm ci
      - name: Lint
        working-directory: backend
```

```
        run: npm run lint --if-present
      - name: Tests
        working-directory: backend
        env:
          DATABASE_URL: postgres://postgres:postgres@localhost:5432/postgres
        run: npm test --if-present


name: Backend CD — build, push, deploy to ECS

on:
  push:
    branches: [ main ]

env:
  IMAGE_TAG: ${{ github.sha }}

jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Login to AWS
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ secrets.AWS_REGION }}

      - name: Configure Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Login to ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      - name: Build and push image to ECR
        uses: docker/build-push-action@v4
        with:
          context: backend
          push: true
          tags: ${{ secrets.ECR_REGISTRY }}/${{ secrets.ECR_REPOSITORY }}:${{
env.IMAGE_TAG }}
          cache-from: type=registry,ref=${{ secrets.ECR_REGISTRY }}/${{
secrets.ECR_REPOSITORY }}:cache
          cache-to: type=inline
```

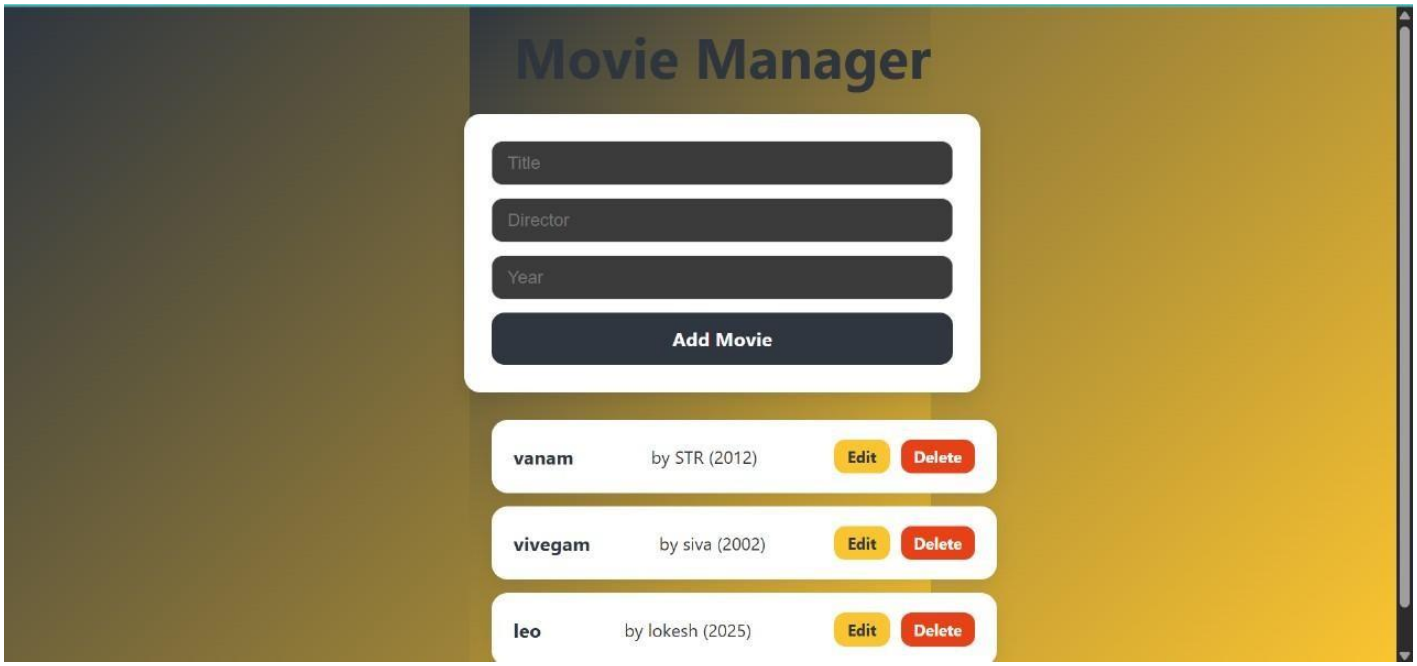
```
      - name: Update ECS task and service
        uses: aws-actions/amazon-ecs-deploy-task-definition@v2
        with:
          task-definition: path/to/ecs-task-def.json
          service: ${{ secrets.ECS_SERVICE }}
          cluster: ${{ secrets.ECS_CLUSTER }}
          wait-for-service-stability: true
```

name: CI — frontend & backend tests

```
on:
  pull_request:
  push:
    branches: [ develop, feature/** ]

jobs:
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: node-version: 20
      - name: Install deps
        working-directory: frontend
        run: npm ci
      - name: Lint
        working-directory: frontend
        run: npm run lint --if-present
      - name: Tests
        working-directory: frontend
        run: npm test --if-present -----watchAll=false

  backend:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: postgres
        POSTGRES_PASSWORD: postgres
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
```

```
  with: node-version: 20
- name: Install deps
  working-directory: backend
  run: npm ci
- name: Lint
  working-directory: backend
  run: npm run lint --if-present
- name: Tests
  working-directory: backend
  env:
    DATABASE_URL: postgres://postgres:postgres@localhost:5432/postgres
  run: npm test --if-present
```

OUTPUT:



RESULT: